

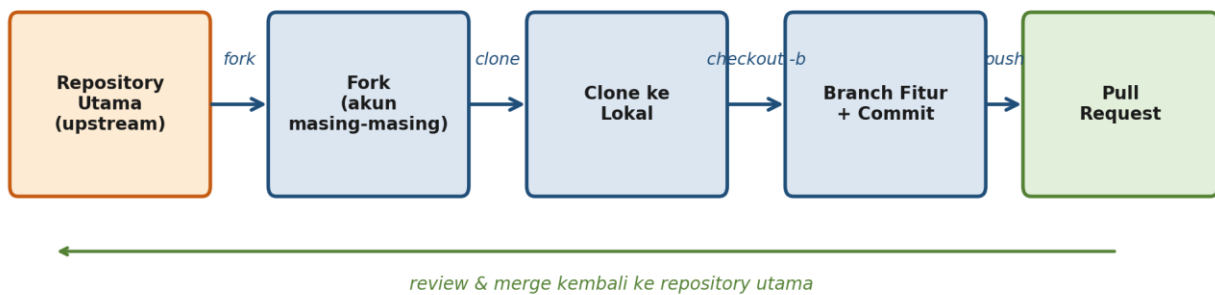
MODUL LANJUTAN

Kolaborasi Tim dengan Git dan GitHub

Pull Request, Code Review, Alur Kerja Tim, dan Konvensi Commit

Kelanjutan dari Modul: Instalasi JDK & Visual Studio Code serta Tutorial Git

Alur Fork, Pull Request, dan Code Review



Daftar Isi

1. Pengantar: Dari Kerja Individu ke Kerja Tim
2. Fork dan Pull Request
3. Code Review di GitHub
4. GitHub Issues dan Projects
5. Alur Kerja Tim: Feature Branch Workflow
6. Strategi Merge: Merge Commit, Squash, dan Rebase
7. Konvensi Pesan Commit (Conventional Commits)
8. Memperbaiki Kesalahan: Revert, Reset, dan Cherry-pick
9. Alat Bantu GUI: GitHub Desktop dan Git Graph
10. Troubleshooting Umum
11. Rangkuman Istilah dan Perintah
12. Jobsheet Praktikum: Simulasi Kolaborasi Tim via Pull Request
13. Daftar Referensi

1. Pengantar: Dari Kerja Individu ke Kerja Tim

Pada modul sebelumnya, mahasiswa telah mempelajari alur kerja Git secara individu: init, add, commit, branch, merge, push, pull, hingga clone. Pada praktiknya, proyek perangkat lunak jarang dikerjakan sendirian — umumnya melibatkan banyak orang yang bekerja pada bagian kode yang sama.

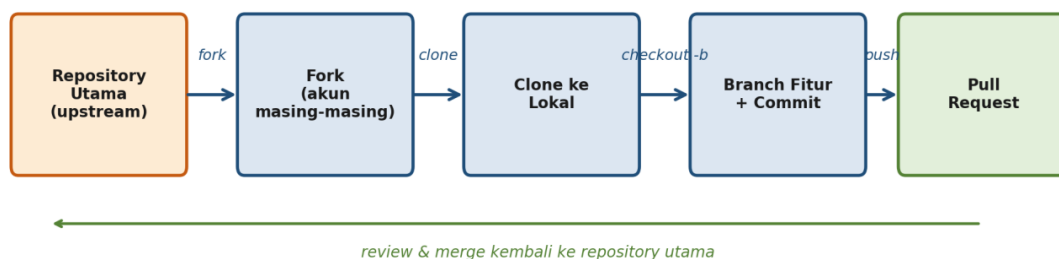
Modul ini membahas bagaimana Git dan GitHub digunakan untuk kolaborasi tim secara terstruktur, mencakup mekanisme Pull Request, proses tinjauan kode (code review), alur kerja branch yang umum dipakai industri, serta kebiasaan menulis pesan commit yang baik.

2. Fork dan Pull Request

Fork adalah proses menyalin sebuah repository ke akun GitHub milik sendiri. Fork umumnya digunakan ketika seseorang ingin berkontribusi pada proyek yang bukan miliknya sendiri, atau ketika tim ingin setiap anggota memiliki salinan repository yang independen.

Pull Request (PR) adalah permintaan resmi untuk menggabungkan perubahan dari sebuah branch (baik dari fork maupun dari branch lain pada repository yang sama) ke branch utama. PR memungkinkan perubahan ditinjau terlebih dahulu sebelum benar-benar digabungkan.

Alur Fork, Pull Request, dan Code Review



2.1 Langkah Umum Membuat Pull Request

Langkah	Keterangan
1	Fork repository target melalui tombol "Fork" pada halaman GitHub repository tersebut (atau, jika bekerja pada repository tim yang sama, cukup buat branch baru tanpa fork).
2	Clone hasil fork ke komputer lokal menggunakan git clone.
3	Buat branch baru khusus untuk perubahan yang akan dikerjakan, misalnya git checkout -b fitur-login.
4	Lakukan perubahan, commit, lalu push branch tersebut ke repository (git push origin fitur-login).

Langkah	Keterangan
5	Buka halaman GitHub, klik "Compare & pull request", isi judul dan deskripsi PR yang menjelaskan perubahan yang dilakukan, lalu klik "Create pull request".
6	Tunggu proses tinjauan (review) dari pemilik repository atau anggota tim lain sebelum PR digabungkan (merge).

Catatan: Fork biasanya dipakai untuk kontribusi ke proyek open source milik orang lain. Untuk proyek tim/kelompok yang sama-sama memiliki akses, cukup gunakan branch baru pada repository yang sama tanpa perlu fork.

3. Code Review di GitHub

Code review adalah proses meninjau perubahan kode yang diajukan melalui Pull Request sebelum digabungkan ke branch utama. Tujuannya adalah menjaga kualitas kode, menemukan potensi bug lebih awal, serta berbagi pengetahuan antar anggota tim.

3.1 Fitur Review pada GitHub

Fitur	Fungsi
Files changed	Menampilkan seluruh perubahan baris kode (diff) pada Pull Request.
Comment	Memberi komentar umum pada Pull Request secara keseluruhan.
Add a comment pada baris kode	Memberi catatan spesifik pada baris tertentu yang perlu didiskusikan.
Approve	Menyetujui perubahan sehingga PR siap digabungkan.
Request changes	Meminta perbaikan sebelum PR dapat digabungkan.
Merge pull request	Menggabungkan perubahan ke branch utama setelah disetujui.

3.2 Etika dan Kebiasaan Baik saat Code Review

- Fokus pada kode, bukan pada personal — sampaikan masukan secara konstruktif.
- Jelaskan alasan di balik sebuah masukan, bukan sekadar menyatakan "salah" atau "kurang tepat".
- Berikan pujian pada bagian kode yang sudah baik, tidak hanya menyoroti kekurangan.
- Jaga agar satu Pull Request tidak terlalu besar, agar lebih mudah ditinjau secara menyeluruh.

4. GitHub Issues dan Projects

Selain Pull Request, GitHub menyediakan fitur untuk mengelola tugas dan progres tim secara terstruktur, yaitu Issues dan Projects. Kedua fitur ini sangat membantu terutama untuk proyek kelompok, agar setiap anggota tahu tugas apa yang perlu dikerjakan dan sejauh mana progresnya.

4.1 GitHub Issues

Issue adalah catatan pada GitHub yang digunakan untuk mencatat tugas, permintaan fitur, laporan bug, atau catatan diskusi terkait proyek. Setiap Issue memiliki nomor unik dan dapat diberi label, ditugaskan (assign) ke anggota tertentu, serta didiskusikan melalui kolom komentar.

Langkah	Keterangan
1	Buka tab "Issues" pada halaman repository GitHub, lalu klik "New issue".
2	Isi judul yang singkat dan jelas, serta deskripsi tugas/masalah pada kolom yang tersedia.
3	Gunakan "Labels" untuk mengkategorikan Issue, misalnya bug, enhancement, atau documentation.
4	Gunakan "Assignees" untuk menentukan anggota tim yang bertanggung jawab menyelesaikan Issue tersebut.
5	Klik "Submit new issue" untuk menyimpan.

4.2 Menghubungkan Pull Request dengan Issue

Sebuah Pull Request dapat dihubungkan secara otomatis dengan Issue tertentu menggunakan kata kunci tertentu pada deskripsi commit atau Pull Request. Ketika PR tersebut digabungkan (merge), Issue terkait akan otomatis tertutup (closed).

```
Closes #12  
Fixes #7  
Resolves #20
```

Contoh penggunaan pada pesan commit:

```
git commit -m "feat: menambahkan validasi form login (Closes #12)"
```

4.3 GitHub Projects (Papan Kanban)

GitHub Projects menyediakan papan visual bergaya Kanban untuk memantau progres seluruh Issue dan Pull Request dalam sebuah proyek, biasanya terbagi menjadi beberapa kolom seperti Todo, In Progress, dan Done.

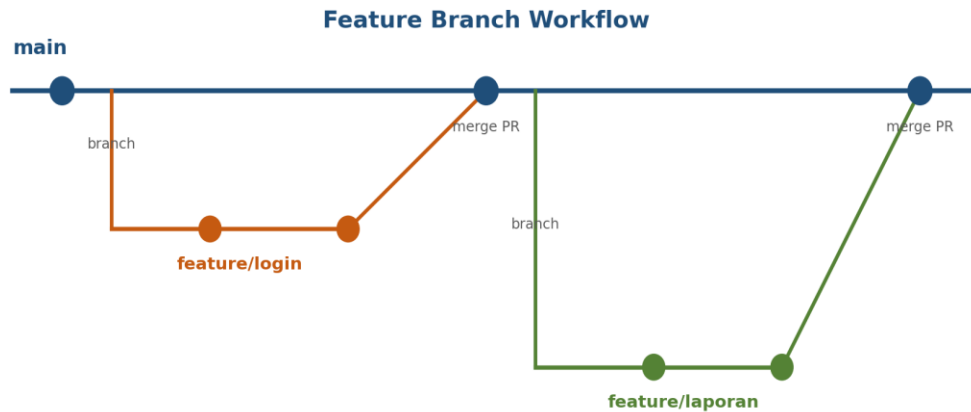
Langkah	Keterangan
1	Buka tab "Projects" pada halaman repository, lalu klik "New project" dan pilih template "Board".

Langkah	Keterangan
2	Tambahkan Issue atau Pull Request yang relevan ke dalam papan menggunakan tombol "Add item".
3	Pindahkan kartu antar kolom (Todo, In Progress, Done) sesuai progres pengerjaan — dapat dilakukan dengan drag and drop.
4	Gunakan papan ini sebagai acuan bersama saat rapat progres kelompok, agar seluruh anggota tim mengetahui status pekerjaan masing-masing.

Tips untuk Kelompok: Biasakan membuat satu Issue untuk setiap tugas sebelum mulai membuat branch dan Pull Request. Ini membuat riwayat proyek lebih mudah ditelusuri: setiap perubahan kode dapat ditelusuri kembali ke alasan/tugas awalnya.

5. Alur Kerja Tim: Feature Branch Workflow

Feature Branch Workflow adalah pola kerja di mana setiap fitur, perbaikan, atau tugas baru dikerjakan pada branch terpisah, lalu digabungkan ke branch utama (main/master) melalui Pull Request setelah selesai dan ditinjau.



5.1 Konvensi Penamaan Branch

Pola Nama	Kegunaan
feature/nama-fitur	Branch untuk pengembangan fitur baru, misalnya feature/login atau feature/laporan-nilai.
fix/nama-perbaikan	Branch untuk perbaikan bug, misalnya fix/validasi-form.
docs/nama-dokumen	Branch untuk perubahan dokumentasi.
hotfix/nama-perbaikan	Branch untuk perbaikan mendesak pada versi yang sudah berjalan di produksi.

5.2 Prinsip Dasar yang Perlu Dipegang

- Branch main/master selalu dalam kondisi yang stabil dan siap digunakan.
- Setiap pekerjaan baru dimulai dari branch main/master yang sudah ter-update (lakukan git pull terlebih dahulu).
- Satu branch idealnya berfokus pada satu tugas/fitur agar mudah ditinjau dan digabungkan.
- Branch yang sudah digabungkan (merged) sebaiknya dihapus agar repository tetap rapi.

Istilah: Gitflow adalah versi yang lebih formal dari alur kerja ini, menambahkan branch permanen seperti develop, release, dan hotfix. Untuk proyek kelas/kelompok kecil, Feature Branch Workflow yang lebih sederhana ini umumnya sudah mencukupi.

6. Strategi Merge: Merge Commit, Squash, dan Rebase

Saat menggabungkan sebuah Pull Request di GitHub, tersedia tiga pilihan strategi penggabungan yang akan memengaruhi bentuk riwayat commit pada branch utama. Memilih strategi yang tepat penting agar riwayat proyek tetap mudah dibaca.

6.1 Perbandingan Tiga Strategi Merge

Strategi	Penjelasan
Merge commit	Menggabungkan seluruh commit dari branch fitur apa adanya, ditambah satu commit gabungan (merge commit) baru. Riwayat lengkap dan menunjukkan kapan sebuah fitur digabungkan, namun bisa terlihat ramai apabila commit pada branch fitur sangat banyak.
Squash and merge	Menggabungkan seluruh commit pada branch fitur menjadi satu commit tunggal pada branch utama. Riwayat menjadi ringkas dan rapi — satu Pull Request setara satu commit — cocok untuk branch fitur dengan banyak commit kecil ("typo", "perbaikan lagi", dll).
Rebase and merge	Memindahkan seluruh commit dari branch fitur ke atas branch utama tanpa membuat commit gabungan tambahan, sehingga riwayat terlihat linear (lurus), seolah-olah dikerjakan berurutan.

Rekomendasi untuk Pemula: Squash and merge umumnya paling aman dan mudah dipahami mahasiswa pemula, karena riwayat pada branch utama tetap ringkas satu Pull Request satu commit, tanpa perlu memahami mekanisme rebase secara mendalam terlebih dahulu.

6.2 Dasar git rebase

git rebase memindahkan commit dari satu branch ke atas titik terbaru branch lain, sehingga riwayat menjadi linear tanpa commit gabungan. Ini berbeda dengan git merge yang mempertahankan riwayat kedua branch apa adanya.

```
git checkout feature/login
git rebase main
```

Perintah di atas akan memindahkan seluruh commit pada feature/login agar seolah-olah dibuat setelah commit terbaru pada main.

Langkah	Keterangan
1	Pastikan branch main dalam keadaan terbaru dengan git pull terlebih dahulu.
2	Pindah ke branch fitur yang akan di-rebase, lalu jalankan git rebase main.
3	Apabila terjadi konflik, Git akan berhenti pada commit yang bentrok — selesaikan konflik pada berkas terkait, lalu jalankan git add pada berkas tersebut.
4	Lanjutkan proses rebase dengan git rebase --continue, atau batalkan seluruhnya dengan git rebase --abort apabila ingin kembali ke kondisi semula.

Langkah	Keterangan
5	Setelah rebase selesai, push branch dengan git push --force-with-lease (karena riwayat commit berubah, push biasa akan ditolak).

Peringatan: Jangan melakukan rebase pada branch yang sudah dibagikan/dipakai bersama anggota tim lain (misalnya branch main), karena akan mengubah riwayat commit dan membingungkan anggota tim lain. Rebase sebaiknya hanya dilakukan pada branch fitur milik sendiri sebelum diajukan sebagai Pull Request.

7. Konvensi Pesan Commit (Conventional Commits)

Pesan commit yang konsisten memudahkan tim memahami riwayat perubahan proyek tanpa perlu membuka setiap berkas satu per satu. Salah satu konvensi yang banyak dipakai adalah Conventional Commits, dengan format umum sebagai berikut:

<code><tipe>: <deskripsi singkat></code> Contoh: feat: menambahkan fitur login mahasiswa fix: memperbaiki validasi input NIM docs: memperbarui panduan instalasi	
Tipe	Keterangan
feat	Menambahkan fitur atau kemampuan baru.
fix	Memperbaiki bug atau kesalahan.
docs	Perubahan pada dokumentasi saja.
style	Perubahan format/penulisan kode tanpa mengubah logika (spasi, indentasi, dll).
refactor	Perubahan struktur kode tanpa mengubah perilaku/fungsi program.
test	Menambahkan atau memperbaiki pengujian (testing).
chore	Pekerjaan pemeliharaan lain, misalnya update dependensi.

Tips: Tulis pesan commit dalam bentuk kalimat perintah aktif dan singkat, misalnya "menambahkan validasi" bukan "menambahkan validasi sudah selesai dikerjakan kemarin malam".

8. Memperbaiki Kesalahan: Revert, Reset, dan Cherry-pick

Kesalahan saat commit atau push adalah hal yang wajar terjadi, termasuk bagi mahasiswa yang baru belajar Git. Bagian ini membahas cara memperbaiki kesalahan tersebut dengan aman.

8.1 git revert — Membatalkan Commit dengan Aman

git revert membuat commit baru yang membalikkan (undo) perubahan dari sebuah commit tertentu, tanpa menghapus riwayat commit yang sudah ada. Ini adalah cara yang paling aman untuk membatalkan perubahan, terutama pada branch yang sudah dibagikan/di-push ke remote.

```
git revert <hash-commit>
```

Karena revert hanya menambahkan commit baru (bukan menghapus riwayat), perintah ini aman digunakan meskipun commit yang ingin dibatalkan sudah di-push ke GitHub.

8.2 git reset — Mengembalikan ke Commit Tertentu

git reset memindahkan branch aktif kembali ke commit tertentu. Berbeda dengan revert, reset dapat menghapus riwayat commit setelah titik yang dituju, sehingga perlu digunakan dengan lebih hati-hati.

Perintah	Efek
git reset --soft <hash-commit>	Kembali ke commit tertentu, namun seluruh perubahan tetap berada di staging area (siap di-commit ulang).
git reset --mixed <hash-commit>	Kembali ke commit tertentu, perubahan tetap ada di working directory namun keluar dari staging area (perilaku default).
git reset --hard <hash-commit>	Kembali ke commit tertentu dan menghapus seluruh perubahan setelahnya secara permanen.

Peringatan: git reset --hard akan menghapus perubahan secara permanen dan tidak dapat dikembalikan dengan mudah. Jangan gunakan git reset pada commit yang sudah di-push dan dipakai bersama anggota tim lain — gunakan git revert untuk kasus tersebut.

8.3 git cherry-pick — Mengambil Satu Commit dari Branch Lain

git cherry-pick digunakan untuk mengambil satu commit tertentu dari branch lain dan menerapkannya ke branch yang sedang aktif, tanpa perlu menggabungkan seluruh branch tersebut.

```
git checkout main  
git cherry-pick <hash-commit>
```

Perintah ini berguna misalnya ketika sebuah perbaikan bug penting sudah dibuat pada branch fitur yang belum selesai sepenuhnya, namun perbaikan tersebut perlu segera diterapkan juga ke branch main.

9. Alat Bantu GUI: GitHub Desktop dan Git Graph

Bagi mahasiswa yang belum terbiasa dengan command line, tersedia alat bantu visual (GUI) yang mempermudah operasi Git sehari-hari tanpa mengetik perintah secara manual.

9.1 GitHub Desktop

GitHub Desktop adalah aplikasi resmi dari GitHub yang menyediakan antarmuka visual untuk clone, commit, push, pull, branch, dan bahkan membuat Pull Request langsung dari aplikasi, tanpa perlu membuka terminal.

Langkah	Keterangan
1	Unduh dan instal GitHub Desktop melalui https://desktop.github.com/
2	Login menggunakan akun GitHub.
3	Pilih "Clone a repository" untuk menyalin repository yang sudah ada, atau "Add local repository" untuk repository yang sudah dibuat sebelumnya di komputer.
4	Perubahan pada berkas akan otomatis terdeteksi dan ditampilkan; isi ringkasan commit pada kolom yang tersedia, lalu klik "Commit to main".
5	Klik "Push origin" untuk mengunggah perubahan, atau "Fetch origin" untuk memeriksa dan menarik pembaruan dari GitHub.

9.2 Ekstensi Git Graph pada VS Code

Git Graph adalah ekstensi VS Code yang menampilkan riwayat commit dan percabangan (branch) secara visual dalam bentuk grafik, sehingga lebih mudah dipahami dibandingkan membaca output git log --graph pada terminal.

Langkah	Keterangan
1	Buka tab Extensions pada VS Code (Ctrl+Shift+X).
2	Cari ekstensi bernama "Git Graph", lalu klik Install.
3	Setelah terpasang, buka Command Palette (Ctrl+Shift+P), ketik "Git Graph: View Git Graph", lalu Enter.
4	Riwayat commit dan percabangan akan ditampilkan secara visual; klik pada sebuah commit untuk melihat detail perubahannya.

10. Troubleshooting Umum

10.1 Pull Request Menampilkan "This branch has conflicts"

Ini berarti terjadi merge conflict antara branch PR dengan branch utama. Tarik dulu pembaruan terbaru dari branch utama ke branch fitur, selesaikan konflik secara lokal, lalu push kembali:

```
git checkout fitur-login
```

```
git pull origin main
# selesaikan konflik pada berkas yang bentrok
git add .
git commit -m "fix: menyelesaikan konflik dengan main"
git push origin fitur-login
```

10.2 Pull Request Ditolak (Request Changes)

Lakukan perbaikan sesuai masukan reviewer pada branch yang sama, lalu commit dan push kembali — PR yang sudah dibuat akan otomatis diperbarui tanpa perlu membuat PR baru.

10.3 Salah Push ke Branch main Secara Langsung

Sebisa mungkin hindari push langsung ke branch main pada proyek tim. Apabila hal ini terjadi secara tidak sengaja dan repository memiliki proteksi branch (branch protection), GitHub akan menolak push tersebut secara otomatis.

11. Rangkuman Istilah dan Perintah

Istilah	Keterangan
Fork	Menyalin repository orang lain ke akun GitHub sendiri.
Pull Request (PR)	Permintaan untuk menggabungkan perubahan dari satu branch ke branch lain.
Code Review	Proses meninjau perubahan kode sebelum digabungkan.
Approve / Request changes	Menyetujui atau meminta perbaikan pada sebuah Pull Request.
Issue	Catatan untuk tugas, permintaan fitur, atau laporan bug pada GitHub.
GitHub Projects	Papan Kanban untuk memantau progres Issue dan Pull Request.
Feature Branch Workflow	Pola kerja: satu branch untuk satu fitur/tugas, digabungkan melalui PR.
Merge commit / Squash / Rebase	Tiga strategi penggabungan Pull Request dengan efek riwayat commit yang berbeda.
git rebase	Memindahkan commit ke atas branch lain agar riwayat menjadi linear.
Conventional Commits	Konvensi penulisan pesan commit dengan format <tipe>: <deskripsi>.
git revert	Membatalkan commit dengan aman melalui commit baru, tanpa menghapus riwayat.
git reset	Mengembalikan branch ke commit tertentu, dapat menghapus riwayat setelahnya.
git cherry-pick	Mengambil satu commit tertentu dari branch lain.

Istilah	Keterangan
Branch protection	Pengaturan pada GitHub yang mencegah push langsung ke branch tertentu.
GitHub Desktop	Aplikasi GUI resmi untuk operasi Git tanpa command line.
Git Graph	Ekstensi VS Code untuk memvisualisasikan riwayat commit dan branch.

12. Jobsheet Praktikum: Simulasi Kolaborasi Tim via Pull Request

A. Tujuan Praktikum

- Mahasiswa mampu membuat branch fitur dan mengajukan Pull Request.
- Mahasiswa mampu melakukan code review terhadap Pull Request rekan sekelompok.
- Mahasiswa mampu menyelesaikan konflik yang muncul pada Pull Request.
- Mahasiswa mampu menulis pesan commit sesuai konvensi Conventional Commits.

B. Alat dan Bahan

- Laptop dengan Git dan VS Code yang sudah terinstal (lihat Modul Dasar).
- Akun GitHub aktif.
- Repository hasil praktikum sebelumnya (jobsheet-git-nim), atau repository baru khusus kegiatan ini.
- Berkelompok dua orang: Mahasiswa A sebagai pemilik repository, Mahasiswa B sebagai kontributor/reviewer.

C. Langkah Kerja

Tahap 1 — Persiapan (Mahasiswa A)

Langkah	Keterangan
1	Pastikan repository jobsheet-git-nim sudah bersifat Public di GitHub.
2	Tambahkan Mahasiswa B sebagai Collaborator melalui menu Settings > Collaborators pada halaman repository (atau minta Mahasiswa B melakukan fork apabila tidak ditambahkan sebagai collaborator).
3	Buat sebuah Issue baru pada tab "Issues" dengan judul "Tambahkan berkas motivasi belajar", beri label enhancement, dan assign ke Mahasiswa B. Catat nomor Issue yang muncul (misalnya #1).

Tahap 2 — Membuat Branch dan Pull Request (Mahasiswa B)

Langkah	Keterangan
4	Clone repository Mahasiswa A menggunakan git clone <url-repository>.
5	Buat branch baru dengan penamaan sesuai konvensi, misalnya git checkout -b feature/tambah-motivasi.
6	Buat berkas baru bernama motivasi.txt berisi satu kalimat motivasi belajar, lalu commit dengan pesan mengikuti Conventional Commits dan menyertakan nomor Issue, misalnya "feat: menambahkan berkas motivasi belajar (Closes #1)".

Langkah	Keterangan
7	Push branch tersebut ke repository menggunakan git push origin feature/tambah-motivasi.
8	Buka halaman GitHub, buat Pull Request dari branch feature/tambah-motivasi ke branch utama, lengkap dengan judul, deskripsi singkat, dan pastikan referensi "Closes #1" tercantum agar Issue otomatis tertutup saat PR digabungkan.

Tahap 3 — Code Review (Mahasiswa A)

Langkah	Keterangan
9	Buka Pull Request yang diajukan Mahasiswa B, periksa tab "Files changed".
10	Berikan minimal satu komentar pada baris kode/berkas tertentu.
11	Apabila perubahan sudah sesuai, klik "Approve"; apabila masih perlu perbaikan, pilih "Request changes" disertai penjelasan.

Tahap 4 — Simulasi Konflik dan Penyelesaiannya

Langkah	Keterangan
12	Mahasiswa A mengubah baris pertama pada berkas profil.txt langsung di branch utama, lalu commit dan push.
13	Mahasiswa B mengubah baris yang sama pada profil.txt di branch feature/tambah-motivasi, commit, lalu push ke branch tersebut.
14	Pada halaman Pull Request, GitHub akan menampilkan status "This branch has conflicts that must be resolved".
15	Mahasiswa B menjalankan git pull origin main pada branch fitur untuk menarik perubahan terbaru, menyelesaikan konflik secara lokal di VS Code, lalu commit dan push kembali.
16	Pastikan status Pull Request kembali menjadi "Able to merge".

Tahap 5 — Merge dengan Strategi Squash, dan Verifikasi Issue

Langkah	Keterangan
17	Setelah disetujui (approved) dan konflik selesai, Mahasiswa A membuka opsi dropdown pada tombol merge, lalu memilih strategi "Squash and merge" (bukan merge commit biasa).
18	Amati bahwa Issue #1 pada tab "Issues" otomatis berubah status menjadi "Closed", sebagai bukti bahwa referensi "Closes #1" bekerja dengan benar.

Langkah	Keterangan
19	Kedua mahasiswa menjalankan git pull origin main pada masing-masing repository lokal untuk menyinkronkan hasil akhir.
20	Verifikasi menggunakan git log --oneline --graph --all — amati bahwa riwayat pada branch utama tampak ringkas (satu commit hasil squash), berbeda dengan riwayat merge commit biasa.

Tahap 6 — Simulasi Perbaikan Kesalahan dengan git revert

Langkah	Keterangan
21	Mahasiswa A membuat sebuah commit baru yang sengaja salah, misalnya menghapus isi berkas profil.txt, lalu commit dan push.
22	Jalankan git log --oneline untuk menemukan hash dari commit yang salah tersebut.
23	Jalankan git revert <hash-commit> untuk membatalkan perubahan tersebut secara aman, lalu push kembali.
24	Verifikasi dengan git log --oneline bahwa muncul commit baru hasil revert, dan isi profil.txt sudah kembali seperti semula.

D. Bukti Pengerjaan yang Dikumpulkan

- Tautan (link) Pull Request yang sudah berstatus "Merged" di GitHub.
- Tangkapan layar percakapan/komentar code review pada Pull Request.
- Tangkapan layar status konflik sebelum diselesaikan, dan status "Able to merge" setelah diselesaikan.
- Tangkapan layar Issue yang dibuat, beserta status "Closed" otomatis setelah Pull Request digabungkan.
- Tangkapan layar pemilihan strategi "Squash and merge" beserta hasil git log --oneline --graph --all yang menunjukkan riwayat ringkas.
- Tangkapan layar proses git revert beserta hasil git log --oneline yang menunjukkan commit hasil revert.

E. Rubrik Penilaian

Perintah	Fungsi
Pembuatan Issue & branch	Issue dibuat dengan judul, label, dan assignee yang sesuai; branch dibuat mengikuti konvensi penamaan.
Pull Request & keterkaitan Issue	Pull Request diajukan dengan deskripsi jelas dan referensi Issue (Closes #) yang berfungsi dengan benar.
Pesan commit	Pesan commit mengikuti format Conventional Commits dan mendeskripsikan perubahan dengan tepat.

Perintah	Fungsi
Code review	Komentar review diberikan secara relevan dan konstruktif pada Pull Request.
Penyelesaian konflik	Konflik berhasil diselesaikan dengan benar tanpa kehilangan perubahan penting dari kedua pihak.
Strategi merge	Squash and merge diterapkan dengan benar dan riwayat commit pada branch utama sesuai hasil yang diharapkan.
Perbaikan kesalahan (revert)	git revert digunakan dengan tepat untuk membatalkan commit yang salah tanpa menghapus riwayat.
Hasil akhir	Pull Request berhasil digabungkan (merged) dan kedua repository lokal tersinkronisasi.

13. Daftar Referensi

- GitHub, Inc. (2024). GitHub Docs — About pull requests. Diakses dari <https://docs.github.com/en/pull-requests>
- GitHub, Inc. (2024). GitHub Docs — About pull request reviews. Diakses dari <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests>
- GitHub, Inc. (2024). GitHub Docs — About issues. Diakses dari <https://docs.github.com/en/issues>
- GitHub, Inc. (2024). GitHub Docs — About GitHub Projects. Diakses dari <https://docs.github.com/en/issues/planning-and-tracking-with-projects>
- GitHub, Inc. (2024). GitHub Docs — About merge methods on GitHub. Diakses dari <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/incorporating-changes-from-a-pull-request/about-merge-methods-on-github>
- GitHub, Inc. (2024). GitHub Docs — About protected branches. Diakses dari <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository>
- GitHub Desktop. (2024). GitHub Desktop Documentation. Diakses dari <https://docs.github.com/en/desktop>
- Git. (2024). Git — git-rebase Documentation. Diakses dari <https://git-scm.com/docs/git-rebase>
- Git. (2024). Git — git-revert Documentation. Diakses dari <https://git-scm.com/docs/git-revert>
- Git. (2024). Git — git-reset Documentation. Diakses dari <https://git-scm.com/docs/git-reset>
- Git. (2024). Git — git-cherry-pick Documentation. Diakses dari <https://git-scm.com/docs/git-cherry-pick>
- Conventional Commits. (2024). Conventional Commits v1.0.0 Specification. Diakses dari <https://www.conventionalcommits.org/>
- Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. Tersedia daring di <https://git-scm.com/book>